

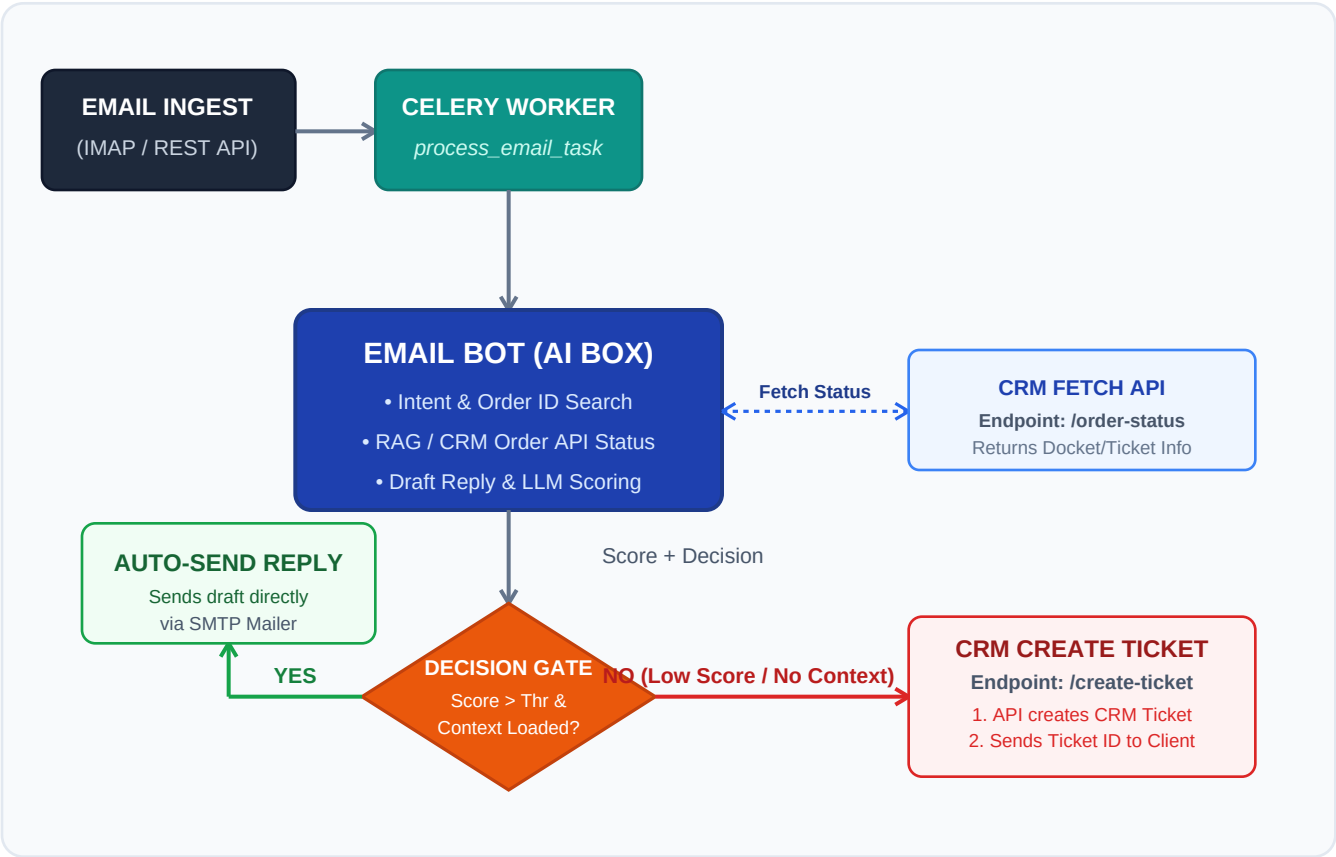
AI EMAIL BOT ARCHITECTURE

System Architecture & Decision Engine Flow Specification

Executive Summary: The AI Email Bot leverages Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) to automate email support. By intercepting incoming emails and evaluating response confidence against a client-specific dynamic threshold, the system acts as an intelligent first responder. It autonomously replies to high-confidence requests while seamlessly escalating low-confidence and missing-context queries to the CRM system by creating formal support tickets.

1. System Flow & Decision State Machine Diagram

The following diagram illustrates how the system ingests data, queries RAG or CRM Status APIs based on intent, evaluates draft responses via LLM scoring, and either triggers direct mail delivery or scales up to a CRM Ticket via a customized JSON-over-Base64 HTTP webhook call.



2. Step-by-Step Processing Pipeline

- Step 1: Data Ingestion:** Incoming emails are fetched continuously from the registered email inbox via an IMAP listener, or pushed via a FastAPI REST endpoint (POST /process-email). These details are packed and queued asynchronously into Redis, then consumed by a background Celery task (process_email_task).
- Step 2: Email Bot AI Box (Abstracted):** Inside the email bot processing unit: **1) Intent Detection:** The LLM parses the query to extract the intent and check for ticket or docket numbers. **2) Context Loading:** If asking for ticket status, it calls the CRM Fetch Ticket Status API; otherwise, it falls back to querying the ChromaDB RAG vector space. **3) Draft Generation & Scoring:** The LLM generates a drafted response based on the fetched context and scores the draft (0-100) based on

relevance.

- **Step 3: Confidence Score & Threshold Gate:** The Celery worker dynamically fetches the client-specific confidence threshold (e.g. 80%). If RAG / API returned no context (**Hard Floor Trigger**) or the generated response score is below or equal to the threshold, the bot escalates. If the score is higher than the threshold and context succeeded, it routes to auto-send.

- **Step 4: Response Actions: Path A (Auto-Send):** The system sends the high-confidence reply directly to the customer via SMTP and marks status as sent.

Path B (CRM Ticket Escalation): The system executes the *CRM Create Ticket Webhook*. The webhook generates a unique CRM Ticket ID (e.g., T-260526-00123). The local database records this ticket. A separate, formatted ticket notification is composed (e.g., 'Ticket T-123 Created') and sent to the client via email.

3. Webhook & API Integrations

To fetch order/ticket status and escalate low-confidence emails, the platform calls custom, client-configured HTTP GET webhooks. These webhooks utilize JSON payloads packed inside Base64 url-encoded variables to interact securely and dynamic with the underlying CRM system.

Active API Endpoints Call Specification

API / Webhook Function	HTTP Method	Dynamic Base URL (Database Loaded)	Query Parameter
Create CRM Ticket Escalates low score/missing context emails to create a ticket in CRM.	GET	db_create_payload_table["url"] e.g., <i>https://crm.c-zentrix.com/api/ticket</i>	?data=<Base64_JSON>
Fetch Ticket Status Checks status of existing ticket using order/docket ID.	GET	get_payload_get_ticket_table["url"] e.g., <i>https://crm.c-zentrix.com/api/status</i>	?postData=<Base64_JSON>

4. Technical Payload Schemas

The following JSON structures illustrate the schema before being Base64-encoded and appended to the respective webhook endpoints.

Create CRM Ticket Webhook Payload (Decoded JSON)	Fetch Ticket Status Webhook Payload (Decoded JSON)
<pre>{ "mail_id": "customer@example.com", // Sender's email "subject": "Reset password request", "body": "I cannot log into my account. Please help.", "status": "Ticket_Generated", "client_specific_key": "client_val" // Pre-configured }</pre>	<pre>{ "filter": { "docket_no": "T-260526-00431" // Ticket / Docket Number }, "client_specific_key": "client_val" // Pre-configured }</pre>

Expected Response Objects

Create CRM Ticket Webhook Response	Fetch Ticket Status Webhook Response
<pre>{ "Status": "Success", "Message": "Ticket created. Reference_No is T-260526-00431" , "Reference_No": "T-260526-00431" }</pre>	<pre>{ "Success": { "ticket_1": { "docket_no": "T-260526-00431", "ticket_status": "Open", "priority_name": "Normal", "ticket_type": "Support", "problem_reported": "I cannot log into my account.", "agent_remarks": "Assigned to tech support", "person": { "person_name": "John Doe", "person_mail": "customer@example.com" } } } }</pre>